

AI ASSISTED CODING LAB ASSIGNMENT 12.1

Name: SIRIVELLA SANJANA

ROLL NO: 2403A510D4

BATCH NO: 05

Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Lab Objectives:

- Apply AI-assisted programming to implement and optimize sorting and searching algorithms.
- Compare different algorithms in terms of efficiency and use

Week6 -

Monday

cases.

- Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

- Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
- Ask AI to include time complexity and space complexity in the function docstring.
- Verify the generated code with test cases.

• Expected Output:

- A functional Python script implementing Merge Sort with proper documentation

Prompt:

generate a Python program that implements the Merge Sort algorithm.

o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order. o Ask AI to include time complexity and space complexity in the function docstring. o Verify the generated code with test cases.give me the code

Code:

```
ai12.1.py > ...
1  def merge_sort(arr: list) -> list:
2      """
3          Sorts a list in ascending order using the Merge Sort algorithm.
4
5          Merge Sort is a divide-and-conquer algorithm. It works by:
6          1. Dividing the unsorted list into n sublists, each containing one element.
7          2. Repeatedly merging sublists to produce new sorted sublists until there is
8             only one sublist remaining. This will be the sorted list.
9
10         Time Complexity:
11         - Best Case:  $O(n \log n)$ 
12         - Average Case:  $O(n \log n)$ 
13         - Worst Case:  $O(n \log n)$ 
14         The time complexity is consistent because the list is always divided
15         in half, and the merge operation takes linear time.
16
17         Space Complexity:
18         -  $O(n)$ 
19         The space complexity is  $O(n)$  because it requires additional space
20         proportional to the size of the input array to store the sub-arrays
21         during the merging process.
22
23         Args:
24         | arr (list): The list of elements to be sorted. Can contain any
25         | | | comparable elements.
26
27         Returns:
28         | list: A new list containing the sorted elements.
29         """
30     if len(arr) <= 1:
```

```

30     if len(arr) <= 1:
31         return arr
32
33     # 1. Divide: Find the midpoint and split the list into two halves.
34     mid = len(arr) // 2
35     left_half = arr[:mid]
36     right_half = arr[mid:]
37
38     # 2. Conquer: Recursively sort both halves.
39     sorted_left = merge_sort(left_half)
40     sorted_right = merge_sort(right_half)
41
42     # 3. Combine: Merge the two sorted halves.
43     sorted_arr = []
44     i = j = 0 # Pointers for left_half and right_half
45
46     while i < len(sorted_left) and j < len(sorted_right):
47         if sorted_left[i] < sorted_right[j]:
48             sorted_arr.append(sorted_left[i])
49             i += 1
50         else:
51             sorted_arr.append(sorted_right[j])
52             j += 1
53
54     # Append any remaining elements from either half
55     sorted_arr.extend(sorted_left[i:])
56     sorted_arr.extend(sorted_right[j:])
57
58     return sorted_arr

```

Ln 110, Col 1 - Spaces: 4 - LHS: 8 - CRLF - (1) Python - 3.12.5 (base) @ C

```

53
54     # Append any remaining elements from either half
55     sorted_arr.extend(sorted_left[i:])
56     sorted_arr.extend(sorted_right[j:])
57
58     return sorted_arr
59
60 # --- Verification with Test Cases ---
61 if __name__ == "__main__":
62     test_cases = [
63         {
64             "name": "Standard case",
65             "input": [8, 3, 5, 1, 9, 4, 7, 2, 6],
66             "expected": [1, 2, 3, 4, 5, 6, 7, 8, 9]
67         },
68         {
69             "name": "Already sorted",
70             "input": [10, 20, 30, 40, 50],
71             "expected": [10, 20, 30, 40, 50]
72         },
73         {
74             "name": "Reverse sorted",
75             "input": [5, 4, 3, 2, 1],
76             "expected": [1, 2, 3, 4, 5]
77         },
78         {
79             "name": "With duplicates",
80             "input": [5, 2, 8, 2, 5, 1],
81             "expected": [1, 2, 2, 5, 5, 8]

```

```

78     {
79         "name": "With duplicates",
80         "input": [5, 2, 8, 2, 5, 1],
81         "expected": [1, 2, 2, 5, 5, 8]
82     },
83     {
84         "name": "With negative numbers",
85         "input": [-5, 10, -15, 0, 5],
86         "expected": [-15, -5, 0, 5, 10]
87     },
88     {
89         "name": "Single element",
90         "input": [42],
91         "expected": [42]
92     },
93     {
94         "name": "Empty list",
95         "input": [],
96         "expected": []
97     }
98 ]
99
100 all_passed = True
101 for i, test in enumerate(test_cases):
102     print(f"--- Running Test Case #{i+1}: {test['name']} ---")
103     print(f"Input:      {test['input']}")
104     actual_output = merge_sort(test['input'])
105     print(f"Output:    {actual_output}")
106     print(f"Expected: {test['expected']}")
107

```

```

107
108     if actual_output == test['expected']:
109         print("Result:  ✅ Passed\n")
110     else:
111         print(f"Result:  ❌ Failed\n")
112         all_passed = False
113
114 print("="*30)
115 if all_passed:
116     print("🎉 All test cases passed successfully! 🎉")
117 else:
118     print("🔥 Some test cases failed. Please review the code. 🔥")
119

```

Output:

```
=====
🎉 All test cases passed successfully! 🎉
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/ai12.1.py"
--- Running Test Case #1: Standard case ---
Input:      [8, 3, 5, 1, 9, 4, 7, 2, 6]
Output:     [1, 2, 3, 4, 5, 6, 7, 8, 9]
Expected:   [1, 2, 3, 4, 5, 6, 7, 8, 9]
Result:     ✅ Passed

--- Running Test Case #2: Already sorted ---
Input:      [10, 20, 30, 40, 50]
Output:     [10, 20, 30, 40, 50]
Expected:   [10, 20, 30, 40, 50]
Result:     ✅ Passed

--- Running Test Case #3: Reverse sorted ---
Input:      [5, 4, 3, 2, 1]
Output:     [1, 2, 3, 4, 5]
Expected:   [1, 2, 3, 4, 5]
Result:     ✅ Passed

--- Running Test Case #4: With duplicates ---
Input:      [5, 2, 8, 2, 5, 1]
Output:     [1, 2, 2, 5, 5, 8]
Expected:   [1, 2, 2, 5, 5, 8]
Result:     ✅ Passed

--- Running Test Case #5: With negative numbers ---
Input:      [-5, 10, -15, 0, 5]
Output:     [-15, -5, 0, 5, 10]
Expected:   [-15, -5, 0, 5, 10]
Result:     ✅ Passed
```

Ln 118, Col 69 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go

```
--- Running Test Case #6: Single element ---
```

```
Input:      [42]
Output:     [42]
Expected:   [42]
Result:     ✅ Passed
```

```
--- Running Test Case #7: Empty list ---
```

```
Input:      []
Output:     []
Expected:   []
Result:     ✅ Passed
```

```
=====
```

```
🎉 All test cases passed successfully! 🎉
```

```
PS C:\Users\DELL\Desktop\vs code\.vscode>
```

Observations:

Okay, I've reviewed the code you provided. Here are some observations:

- **Correctness:** The `merge_sort` function appears to be implemented correctly. It follows the standard divide-and-conquer approach of the Merge Sort algorithm. The test cases also seem to cover a good range of scenarios, including empty lists, already sorted lists, lists with duplicates, and lists with negative numbers.
- **Clarity:** The code is generally well-structured and easy to read. The docstrings are comprehensive, explaining the algorithm, its time and space complexity, and providing examples. The comments within the code also help to clarify the different steps of the algorithm.
- **Efficiency:** Merge Sort has a time complexity of $O(n \log n)$ in all cases (best, average, and worst), which is efficient for sorting algorithms. However, it has a space complexity of $O(n)$, which can be a concern for very large lists.
- **Test Cases:** The test cases are well-organized and provide good coverage. The output for each test case is clear and easy to understand.
- **Error Handling:** The code doesn't explicitly handle any potential errors (e.g., non-comparable elements in the list). However, this is acceptable for a basic implementation of Merge Sort.

Overall, the code is well-written, correct, and easy to understand. There are no major issues or areas for improvement.



Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - Include docstrings explaining best, average, and worst-case complexities.
 - Test with various inputs.
- Expected Output:
 - Python code implementing binary search with AI-generated comments and docstrings

Prompt:

reate a binary search function that finds a target element in a sorted list. • Instructions: ○ Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found. ○ Include docstrings explaining best, average, and worst- case complexities. ○ Test with various inputs.give me the code

Code:

```
ai12.1(2).py > ...
1 def binary_search(arr: list, target: int) -> int:
2     """
3     Finds the index of a target element in a sorted list using binary search.
4
5     Binary search is an efficient algorithm that works on sorted arrays. It
6     repeatedly divides the search interval in half. If the value of the search
7     key is less than the item in the middle of the interval, it narrows the
8     interval to the lower half. Otherwise, it narrows it to the upper half.
9
10    Time Complexity:
11        - Best Case: O(1)
12          This occurs when the target element is found at the very middle of
13          the list in the first comparison.
14        - Average Case: O(log n)
15          On average, the algorithm will need to halve the search space
16          logarithmically until the element is found.
17        - Worst Case: O(log n)
18          This occurs when the target element is not in the list or is found
19          at the end of the search (when the subarray has only one element).
20
21    Space Complexity:
22        - O(1)
23          The algorithm is iterative and uses a fixed number of variables,
24          so it requires constant extra space.
25
26    Args:
27        arr (list): A sorted list of elements to search within.
28        target: The element to search for.
29
30    Returns:
```

```
1 def binary_search(arr: list, target: int) -> int:
29
30    Returns:
31        int: The index of the target element if found, otherwise -1.
32    """
33    low = 0
34    high = len(arr) - 1
35
36    while low <= high:
37        # Calculate the middle index to avoid potential overflow
38        mid = low + (high - low) // 2
39
40        # Check if the target is at the middle
41        if arr[mid] == target:
42            return mid
43        # If target is greater, ignore the left half
44        elif arr[mid] < target:
45            low = mid + 1
46        # If target is smaller, ignore the right half
47        else:
48            high = mid - 1
49
50    # If the loop finishes, the element was not present
51    return -1
52
53    # --- Verification with Test Cases ---
54    if __name__ == "__main__":
55        # A sorted list is required for binary search
56        sorted_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
57
```

```

58     test_cases = [
59         {"name": "Target in the middle", "target": 16, "expected": 4},
60         {"name": "Target at the beginning", "target": 2, "expected": 0},
61         {"name": "Target at the end", "target": 91, "expected": 9},
62         {"name": "Target in lower half", "target": 8, "expected": 2},
63         {"name": "Target in upper half", "target": 56, "expected": 7},
64         {"name": "Target not found (smaller)", "target": 1, "expected": -1},
65         {"name": "Target not found (larger)", "target": 100, "expected": -1},
66         {"name": "Target not found (in between)", "target": 20, "expected": -1},
67         {"name": "Search in an empty list", "list": [], "target": 5, "expected": -1},
68         {"name": "Search in a single-element list (found)", "list": [7], "target": 7, "expected": 0},
69         {"name": "Search in a single-element list (not found)", "list": [7], "target": 5, "expected": -1}
70     ]
71
72     all_passed = True
73     print(f"Base sorted list for most tests: {sorted_list}\n")
74
75     for i, test in enumerate(test_cases):
76         # Use a custom list for the test if provided, otherwise use the default sorted_list
77         current_list = test.get("list", sorted_list)
78         target = test["target"]
79
80         print(f"--- Running Test Case #{i+1}: {test['name']} ---")
81         print(f"Searching for {target} in {current_list}")
82
83         actual_output = binary_search(current_list, target)
84
85         print(f"Output: {actual_output}")
86         print(f"Expected: {test['expected']}")

```

```

87
88         if actual_output == test['expected']:
89             print("Result:  ✅ Passed\n")
90         else:
91             print(f"Result:  ❌ Failed\n")
92             all_passed = False
93
94     print("\n*30)
95     if all_passed:
96         print("🎉 All test cases passed successfully! 🎉")
97     else:
98         print("🔥 Some test cases failed. Please review the code. 🔥")
99
100

```

Output:

🎉 All test cases passed successfully! 🎉

PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/ai12.1(2).py"

Base sorted list for most tests: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

--- Running Test Case #1: Target in the middle ---

Searching for 16 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Output: 4

Expected: 4

Result:  Passed

--- Running Test Case #2: Target at the beginning ---

Searching for 2 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Output: 0

Expected: 0

Result:  Passed

--- Running Test Case #3: Target at the end ---

Searching for 91 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Output: 9

Expected: 9

Result:  Passed

--- Running Test Case #4: Target in lower half ---

Searching for 8 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Output: 2

Expected: 2

Result:  Passed

--- Running Test Case #5: Target in upper half ---

Searching for 56 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]

Output: 7

Expected: 7

```

--- Running Test Case #5: Target in upper half ---
Searching for 56 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Output: 7
Expected: 7
Result:  Passed

--- Running Test Case #6: Target not found (smaller) ---
Searching for 1 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Output: -1
Expected: -1
Result:  Passed

--- Running Test Case #7: Target not found (larger) ---
Searching for 100 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Output: -1
Expected: -1
Result:  Passed

--- Running Test Case #8: Target not found (in between) ---
Searching for 20 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
Output: -1
Expected: -1
Result:  Passed

--- Running Test Case #9: Search in an empty list ---
Searching for 5 in []
Output: -1
Expected: -1
Result:  Passed

--- Running Test Case #10: Search in a single-element list (found) ---
Searching for 7 in [7]
Output: 0

```

```

--- Running Test Case #10: Search in a single-element list (found) ---
Searching for 7 in [7]
Output: 0
Expected: 0
Result:  Passed

--- Running Test Case #11: Search in a single-element list (not found) ---
Searching for 5 in [7]
Output: -1
Expected: -1
Result:  Passed

=====
🎉 All test cases passed successfully! 🎉
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

Observations:

What's Done Well:

1. **Correctness and Efficiency:** The core logic is flawless. It correctly implements the binary search algorithm with $O(\log n)$ time complexity and $O(1)$ space complexity, which is optimal.
2. **Docstrings and Complexity Analysis:** The docstring is comprehensive. It clearly explains the function's purpose, parameters, and return value. The inclusion of best, average, and worst-case time complexities is a fantastic practice that makes the function's performance characteristics immediately clear.
3. **Readability:** The code is clean, well-commented, and uses meaningful variable names (`low`, `high`, `mid`), making it easy to understand.
4. **Robustness:** The calculation `mid = low + (high - low) // 2` is a great choice. It prevents potential integer overflow issues that can occur in other languages with `(low + high) // 2` when dealing with extremely large arrays, which is a sign of a seasoned developer.
5. **Thorough Testing:** The `if __name__ == "__main__":` block provides a robust set of test cases. It covers a wide range of scenarios, including:
 - Elements at the start, middle, and end.
 - Elements not present (smaller, larger, and in-between).
 - Edge cases like an empty list and a single-element list. This testing harness is well-structured and gives high confidence in the function's correctness.

Minor Suggestions for Enhancement:

Minor Suggestions for Enhancement:

While the code is already of high quality, here are a couple of minor suggestions that could make it even more versatile.

1. Generalize Type Hints for Broader Use

The current type hints `arr: list` and `target: int` are specific. The algorithm works for any sorted list of comparable types (like floats or strings), not just integers. You could generalize this using `typing.Sequence` and `typing.TypeVar` for greater flexibility and type safety.

2. Consider Using Python's `bisect` Module for Standard Library Solutions

For production code where you don't need to implement the algorithm from scratch, Python's built-in `bisect` module is highly optimized and can be more concise. It's worth knowing about as an alternative.

- `bisect.bisect_left(a, x)` finds an insertion point for `x` in `a` to maintain sort order. If `x` is already in `a`, the insertion point is before (to the left of) any existing entries.
- This can be used to find the index of an element.

Task Description #3 (Real-Time Application – Inventory Management System)

• Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

• Task:

- Use AI to suggest the most efficient search and sort

algorithms for this use case.

- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

• Expected Output:

- A table mapping operation → recommended algorithm → justification.
- Working Python functions for searching and sorting the inventory.

Prompt:

S

SANJANA SIRIVELLA

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
 2. Sort products by price or quantity for stock analysis.
- Task:
- o Use AI to suggest the most efficient search and sort algorithms for this use case.
 - o Implement the recommended algorithms in Python.
 - o Justify the choice based on dataset size, update frequency, and performance requirements.
- give me the code



Code:

```

ai12.1(3).py > ...
1  from typing import List, Dict, Optional
2
3  class Product:
4      """A simple class to represent a product in the inventory."""
5      def __init__(self, product_id: int, name: str, price: float, quantity: int):
6          self.product_id = product_id
7          self.name = name
8          self.price = price
9          self.quantity = quantity
10
11     def __repr__(self) -> str:
12         """String representation for easy printing."""
13         return (f"Product(ID={self.product_id}, Name='{self.name}', "
14                 f"Price=${self.price:.2f}, Qty={self.quantity})")
15
16     class InventoryManager:
17         """
18         Manages a retail inventory, providing efficient search and sort operations.
19         """
20         def __init__(self, products: List[Product]):
21             # --- O(1) Average Time Complexity Search by ID ---
22             # We use a dictionary (hash map) to store products with their ID as the key.
23             # This allows for near-instantaneous lookups.
24             # The initial creation of this map takes O(n) time.
25             self._products_by_id: Dict[int, Product] = {p.product_id: p for p in products}

```

```

26
27     def __init__(self, products: List[Product]):
28         # --- O(log n) Search by Name ---
29         # For name search, we'll use a list sorted by name to enable binary search.
30         # The initial sort takes O(n log n).
31         self._products_sorted_by_name: List[Product] = sorted(
32             products, key=lambda p: p.name
33         )
34
35     def find_by_id(self, product_id: int) -> Optional[Product]:
36         """
37         Finds a product by its ID using a hash map.
38         Time Complexity: O(1) on average.
39         """
40         print(f"\nSearching for product with ID: {product_id}")
41         return self._products_by_id.get(product_id)
42
43     def find_by_name(self, name: str) -> Optional[Product]:
44         """
45         Finds a product by its name using binary search on a sorted list.
46         Time Complexity: O(log n).
47         """
48         print(f"\nSearching for product with name: '{name}'")
49         low, high = 0, len(self._products_sorted_by_name) - 1

```

```

42     def find_by_name(self, name: str) -> Optional[Product]:
43
44         result = None
45
46         while low <= high:
47             mid = low + (high - low) // 2
48             mid_product = self._products_sorted_by_name[mid]
49
50             if mid_product.name == name:
51                 # In a real system with duplicates, you might continue searching
52                 # left or right. For this example, we return the first one found.
53                 result = mid_product
54                 break
55             elif mid_product.name < name:
56                 low = mid + 1
57             else:
58                 high = mid - 1
59
60         return result
61
62     def get_sorted_by(self, key: str = "price", reverse: bool = False) -> List[Product]:
63         """
64         Returns a new list of all products sorted by a given key.
65         Uses Python's built-in Timsort, which is O(n log n).
66
67         Args:
68             key (str): The attribute to sort by ('price' or 'quantity').
69             reverse (bool): False for ascending, True for descending.
70
71         """
72

```

```

73         print(f"\nSorting all products by '{key}' {'descending' if reverse else 'ascending'}...")
74
75         # Get all products from our primary dictionary
76         all_products = list(self._products_by_id.values())
77
78         # Use Python's highly optimized `sorted()` function (Timsort)
79         if key not in ["price", "quantity"]:
80             raise ValueError("Can only sort by 'price' or 'quantity'.")
81
82         return sorted(all_products, key=lambda p: getattr(p, key), reverse=reverse)
83
84 # --- Verification and Demonstration ---
85 if __name__ == "__main__":
86     # Sample inventory data
87     sample_products = [
88         Product(102, "Laptop", 1200.50, 15),
89         Product(205, "Mouse", 25.00, 150),
90         Product(101, "Keyboard", 75.99, 80),
91         Product(303, "Monitor", 200.00, 45)
92     ]

```

```

91 sample_products = [
92     Product(102, "Laptop", 1200.50, 15),
93     Product(205, "Mouse", 25.00, 150),
94     Product(101, "Keyboard", 75.99, 80),
95     Product(301, "Monitor", 300.00, 45),
96     Product(404, "Webcam", 55.25, 60),
97     Product(210, "USB Hub", 22.50, 120),
98 ]
99
100 # Initialize the manager
101 inventory = InventoryManager(sample_products)
102
103 # 1. Test fast search by ID (O(1))
104 product = inventory.find_by_id(301)
105 print(f"Found: {product}")
106 product_not_found = inventory.find_by_id(999)
107 print(f"Found: {product_not_found}")
108
109 # 2. Test fast search by name (O(log n))
110 product = inventory.find_by_name("Webcam")
111 print(f"Found: {product}")
112 product_not_found = inventory.find_by_name("Microphone")
113 print(f"Found: {product_not_found}")
114
115 # 3. Test sorting by price (O(n log n))

```

```

115 # 3. Test sorting by price (O(n log n))
116 products_by_price = inventory.get_sorted_by("price")
117 for p in products_by_price:
118     print(p)
119
120 # 4. Test sorting by quantity in descending order (O(n log n))
121 products_by_quantity_desc = inventory.get_sorted_by("quantity", reverse=True)
122 for p in products_by_quantity_desc:
123     print(p)
124

```

Output:

```

🚀 All test cases passed successfully! 🚀
> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/
DELL/Desktop/vs code/.vscode/ai12.1(3).py"

Searching for product with ID: 301
Found: Product(ID=301, Name='Monitor', Price=$300.00, Qty=45)

Searching for product with ID: 999
Found: None

Searching for product with name: 'Webcam'
Found: Product(ID=404, Name='Webcam', Price=$55.25, Qty=60)

Searching for product with name: 'Microphone'
Found: None

Sorting all products by 'price' ascending...
Product(ID=210, Name='USB Hub', Price=$22.50, Qty=120)
Product(ID=205, Name='Mouse', Price=$25.00, Qty=150)
Product(ID=404, Name='Webcam', Price=$55.25, Qty=60)
Product(ID=101, Name='Keyboard', Price=$75.99, Qty=80)
Product(ID=301, Name='Monitor', Price=$300.00, Qty=45)
Product(ID=102, Name='Laptop', Price=$1200.50, Qty=15)

Sorting all products by 'quantity' descending...
Product(ID=205, Name='Mouse', Price=$25.00, Qty=150)
Product(ID=210, Name='USB Hub', Price=$22.50, Qty=120)
Product(ID=101, Name='Keyboard', Price=$75.99, Qty=80)
Product(ID=404, Name='Webcam', Price=$55.25, Qty=60)
Product(ID=301, Name='Monitor', Price=$300.00, Qty=45)
Product(ID=102, Name='Laptop', Price=$1200.50, Qty=15)
PS C:\Users\DELL\Desktop\vs code\.vscode>

```

Observations:

Of course! As Gemini Code Assist, I've reviewed the `InventoryManager` implementation. This is a very strong piece of code that correctly identifies and applies efficient algorithms for the specified tasks. Here are my observations and suggestions for potential enhancements.

Code Review and Observations

The design of the `InventoryManager` is excellent. It demonstrates a solid understanding of how to choose the right data structures for different access patterns, which is a critical skill in software engineering.

What's Done Well:

- Algorithm Selection:** The choice of a hash map (`dict`) for $O(1)$ ID-based lookups and a pre-sorted list for $O(\log n)$ name-based binary searches is optimal for read-heavy operations. This is the most performant approach for the stated requirements.
- Leveraging Python's Strengths:** The code wisely uses Python's built-in `sorted()` function for on-demand sorting. This function implements Timsort, a highly optimized and stable sorting algorithm that performs exceptionally well on real-world data, making it a better choice than implementing a classic sort like Quicksort from scratch.
- Code Clarity and Structure:** The code is clean, well-structured, and readable. The use of private attributes (`__products_by_id`) is good practice, and the docstrings clearly explain the purpose and time complexity of each method.
- Robustness:** The `Product` class has a helpful `__repr__` for easy debugging. The `get_sorted_by` method includes a check to validate the `key` argument, preventing `AttributeError` exceptions for invalid sort keys.
- Comprehensive Demonstration:** The `if __name__ == "__main__":` block serves as a great demonstration and a set of integration tests, verifying that each public

5. **Comprehensive Demonstration:** The `if __name__ == "__main__":` block serves as a great demonstration and a set of integration tests, verifying that each public method works as expected.

Suggestions for Enhancement:

The current implementation is excellent for a static inventory. However, a real-world inventory is dynamic. The main challenge with the current design is that adding or removing products requires keeping both the dictionary and the sorted list synchronized. This can be complex and error-prone.

Here are some suggestions to make the class more robust and maintainable for a dynamic inventory.

1. Encapsulate Data Structures and Add Mutator Methods

To handle a dynamic inventory, you should add `add_product` and `remove_product` methods. These methods will ensure that both `_products_by_id` and `_products_sorted_by_name` are kept in a consistent state. To maintain the $O(\log n)$ performance of `find_by_name`, we can use Python's `bisect` module to insert new products into the sorted list in $O(\log n)$ time.

2. Make `Product` Class More Robust

The `Product` class could be enhanced by making it a dataclass and implementing comparison methods (`__lt__`, `__eq__`, etc.). This makes the code more concise and allows products to be compared directly, simplifying sorting logic.

Prompts to try:

Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output